



# Déploiement AWS ECS avec Docker

Guide complet pour déployer une application Spring Boot sur AWS ECS avec RDS MySQL, Load Balancer et Auto Scaling.



## ÉTAPE 1

# Installation et Configuration AWS CLI

01

## Installer AWS CLI

Télécharger depuis [docs.aws.amazon.com/cli](https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html)

02

## Créer un utilisateur IAM

Nom : AWS-CLI-ECR avec politiques  
AmazonEC2ContainerRegistryFullAccess et  
AmazonElasticContainerRegistryPublicFullAccess

03

## Générer clés d'accès

IAM → Utilisateurs → Informations d'identification de sécurité →  
Créer une clé d'accès

04

## Configurer AWS CLI

Exécuter `aws configure` avec Access Key ID, Secret Access Key,  
région (eu-west-2) et format JSON

# Créer le Repository ECR

Amazon Elastic Container Registry (ECR) héberge vos images Docker de manière sécurisée.

- ECR → Repositories → Create repository
- Type : Private
- Nom : springboot-shop-prod
- Créer le repository





# Construire et Pousser l'Image Docker

Suivez les commandes Push depuis ECR pour déployer votre image.



## Se connecter à ECR

`aws ecr get-login-password | docker login`



## Construire l'image

`docker build -t springboot-shop-prod .`



## Taguer l'image

`docker tag` avec l'URL ECR complète



## Pousser vers ECR

`docker push` vers le repository

# Configuration des Security Groups

Définir les règles de sécurité pour contrôler le trafic réseau entre les composants.



## Service ECS

**Nom :** shop-service-SG

**Port :** 8080 / 443

**Source :** 0.0.0.0/0 (Internet)



## Base RDS

**Nom :** shop-bd-SG

**Port :** 3306

**Source :** SG du Service ECS uniquement







ÉTAPE 5

# Créer la Base de Données MySQL RDS

## Configuration Principale

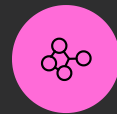
- Engine : MySQL
- Template : Production
- Instance : backend-instance
- Username : admin
- Instance class : db.t3.micro
- Storage : 20 GB (gp2)

## Réseau et Sécurité

- VPC : même que ECS
- Public access : Non
- Subnet : Private subnets
- Security group : shop-bd-SG
- DB initiale : shop\_db

🔗 ÉTAPE 6

# Créer le Cluster et Task Definition ECS



## Cluster ECS

Type : Fargate only

Nom : backend-cluster



## Task Definition

CPU : 1 vCPU

Memory : 2 GB

Port : 8080



## Variables d'environnement

SPRING\_PROFILES\_ACTIVE, DB\_HOST, DB\_PORT, DB\_NAME, DB\_USER,  
DB\_PASS, JWT\_SECRET

# Déployer et Tester le Service ECS

## 1 — Créer le Service

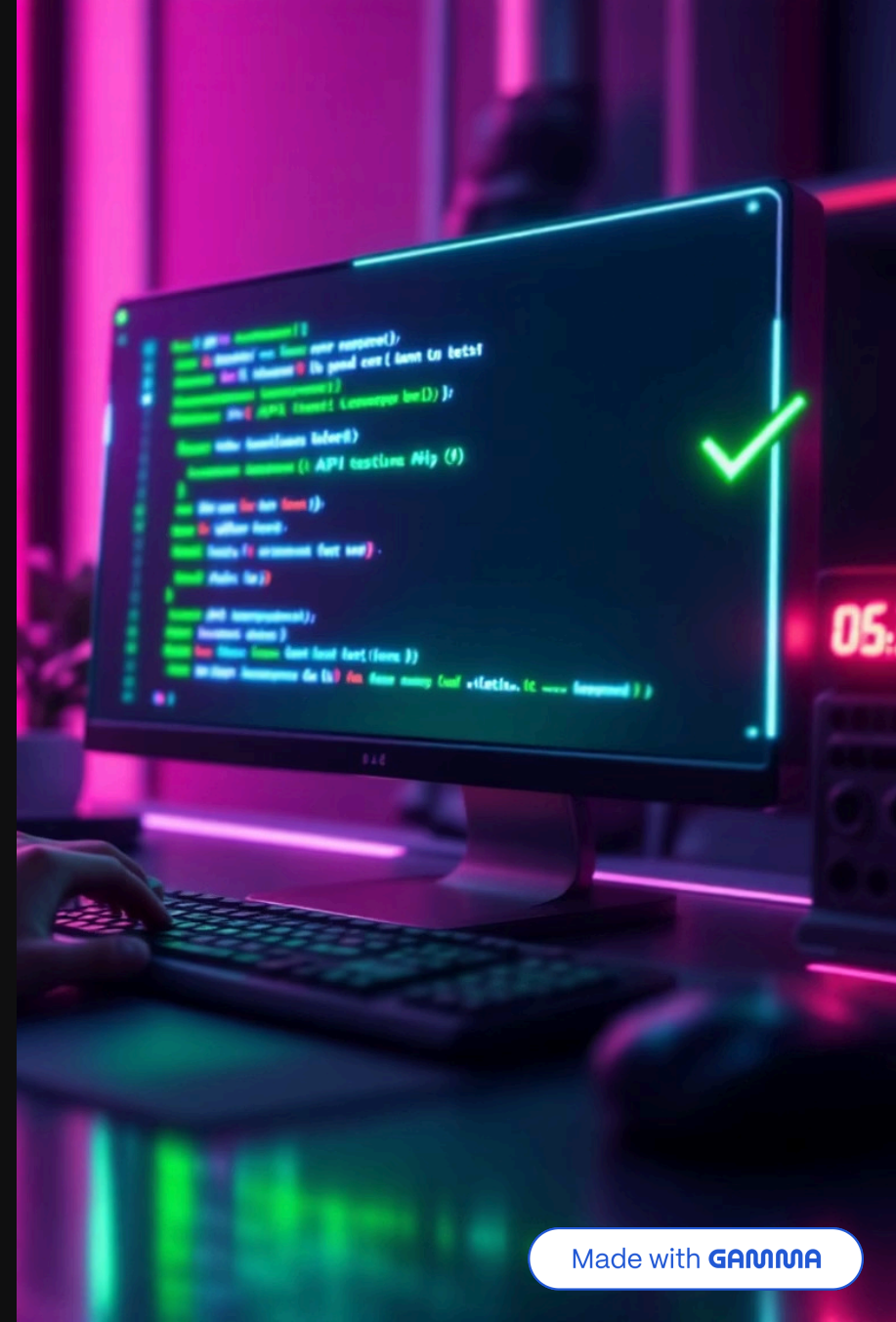
Cluster : backend-cluster, Fargate, 1 tâche souhaitée, VPC et SG configurés

## 2 — Récupérer l'IP

ECS → Cluster → Service → Tâches → IP publique

## 3 — Tester l'API

<http://IP:8080/actuator/health> et endpoints API via Postman





# Load Balancer avec Domaine HTTPS

Configurer un Application Load Balancer avec certificat SSL pour un accès sécurisé via nom de domaine.



## Mettre à jour Security Groups

shop-LB-SG (80/443 depuis Internet), shop-service-SG (8080 depuis LB uniquement)



## Créer Load Balancer

Type : Application LB, listener HTTP 80, target group avec health check /actuator/health, 2 tâches



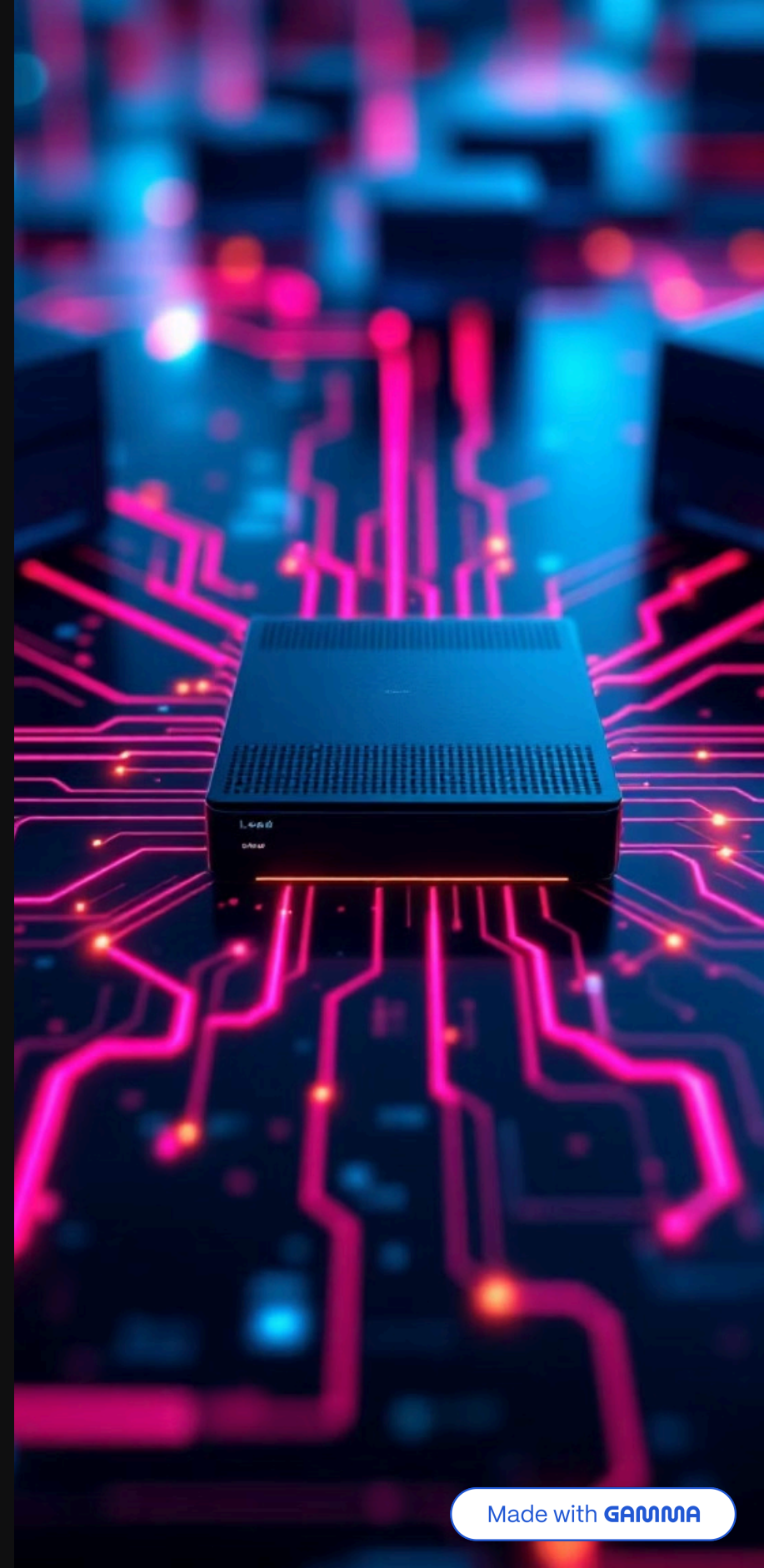
## Certificat SSL (ACM)

Demander certificat public pour domaine, validation DNS via Route 53



## Route 53 et HTTPS

Enregistrement A pointant vers LB, listener HTTPS 443, redirection HTTP→HTTPS



# Auto Scaling pour Haute Disponibilité

Configurer l'auto scaling pour ajuster automatiquement le nombre de tâches selon la charge.

## Configuration

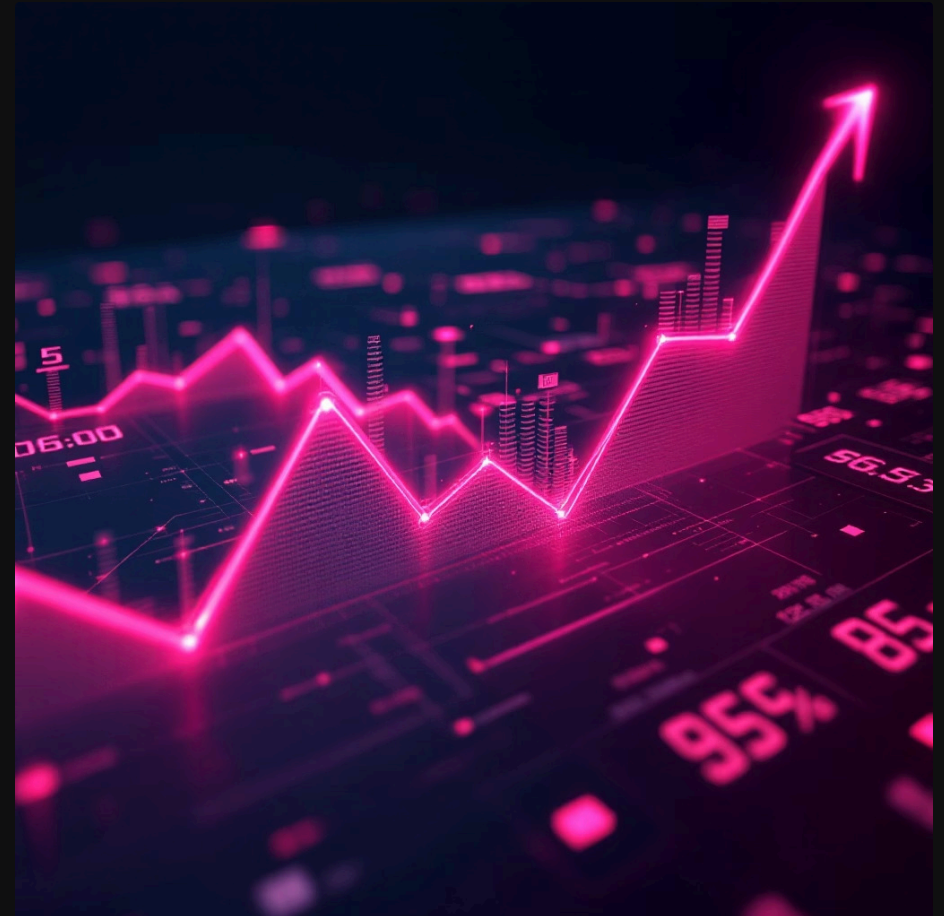
Tâches : min 1, max 3

## Métrique

ALBRequestCountPerTarget

## Politique

springboot-shop-backend-service-autoscaling



📌 **Félicitations !** Votre application Spring Boot est maintenant déployée sur AWS ECS avec haute disponibilité, sécurité SSL et auto scaling.